

Malicious URL Detection System

SYSTEM FLOW

Step 1 — You navigate to a page

Example: you type `https://paypal-security-alert.com/verify` and hit Enter.

Step 2 — background.js fires

Chrome triggers the `tabs.onUpdated` listener the moment the page finishes loading (`status === "complete"`). It grabs the URL from the tab.

Step 3 — POST /predict is called

`background.js` sends the URL to your Flask API:

json

POST `http://localhost:5000/predict`

```
{"url": "https://paypal-security-alert.com/verify"}
```

Step 4 — Flask API processes it

Two-layer check:

- **Layer 1** — Is it in the whitelist (54 trusted domains)?
 - Yes → instantly returns BENIGN, skip model
 - No → go to Layer 2
- **Layer 2** — Build 548-feature vector:
 - Extract 48 heuristic features
 - Scale with `StandardScaler`
 - Transform with char + word TF-IDF vectorizers
 - Feed into Random Forest → get malicious probability
 - Compare against threshold (0.44) → MALICIOUS or BENIGN

Step 5 — Badge updates instantly

`background.js` receives the result and updates the extension icon:

-  ! red badge → MALICIOUS
-  ✓ green badge → BENIGN
-  ? gray badge → API unreachable
- Result is also cached in `chrome.storage.session` by `tabId`.

Step 6 — Warning banner appears (if MALICIOUS)

background.js sends a message to content.js which injects a red warning strip at the top of the page:

⚠ Malicious URL Detected — This page may be dangerous. Confidence: 87.5%

Step 7 — You click the extension icon

popup.js opens and:

- Checks session cache first (no need to call API again)
- Renders the verdict card — MALICIOUS/BENIGN, confidence bar, URL

Step 8 — POST /explain is called

popup.js simultaneously calls:

json

POST http://localhost:5000/explain

```
{"url": "https://paypal-security-alert.com/verify", "num_features": 20}
```

LIME perturbs the feature vector ~5000 times, runs each through the Random Forest, fits a linear model and returns top features.

Step 9 — Explanation renders in popup

After filtering out char_* and word_* features, top 3 human-readable reasons appear:

-  Brand impersonation detected — detected
-  Fake brand domain pattern — detected
-  Suspicious domain ending (.tk .xyz) — detected

**NOTE: BEFORE RUN THE CODES MAKE SURE THAT THE DATA FOLDER ONLY CONTAIN RAW DATA ONLY
WE PROCESS 6 FILES THOSE FILES ARE EXIST**

- Phish.csv
- urlhaus.txt
- openphish.txt
- top-1m.csv
- data.csv
- synthetic_malicious.csv

0. RUN PYTHON SCRIPTS BELOW ORDER

- Generate_typosquatting.py (generate typo squatting pattern URLs)
- Preprocessing.py
- Split.py
- Feature_extraction.py
- Model_train.py
- Model_test.py (only for checking the model before further development no need any more until you add another features (heuristic or NLP))
- Evaluate.py (plot the model additional measures like F1, Accuracy and so on)
- Experiment.py (do experiment with 150 – 300 trees to pick best one estimator in our case 300 perform slightly well compare than others)
- Whitelist.py (to add whitelist domain from the trance dataset)

NOTE : THE FILES EXACT NAMES SOMETIME DIFFER FROM ABOVE MENTIONED

1. CREATE LIME BACKGROUND NPZ

```
cd "D:\Research Project\Research_Working_dir_2\malicious_url_detection"
.\venv\Scripts\activate

python -c "
import sys, os
sys.path.insert(0, 'app')

bg_path = 'models/lime_background.npz'
if os.path.exists(bg_path):
    os.remove(bg_path)
    print('Removed old lime_background.npz')

from model_loader import classifier
print('Building new LIME background...')
classifier._load_background()
print('Done — models/lime_background.npz saved') "
```

2. REBUILD LIME BACKGROUND:

```
python -c "
import sys, os
sys.path.insert(0, 'app')
os.remove('models/lime_background.npz')
from model_loader import classifier
classifier._load_background()
print('Done')
"
```

3. TO RUN THIS SYSTEM

```
cd "D:\Research Project\Research_Working_dir_2\malicious_url_detection"
.\venv\Scripts\activate
python app/app.py
```

```
# ONE-TIME DATA GENERATION
PYTHON SCRIPTS/GENERATE_TYPOSQUATTING.PY
PYTHON SCRIPTS/GENERATE_WHITELIST.PY

# FULL TRAINING PIPELINE
PYTHON SCRIPTS/PREPROCESSING.PY
PYTHON SCRIPTS/SPLIT.PY
PYTHON SCRIPTS/FEATURE_EXTRACTION.PY
PYTHON SCRIPTS/TRAIN_MODEL.PY
PYTHON SCRIPTS/EVALUATE_MODEL.PY

# VERIFY BEFORE PRODUCTION
PYTHON SCRIPTS/TEST_MODEL.PY

# START FLASK
PYTHON APP/APP.PY
```